

Συγκριτική Αξιολόγηση Αλγορίθμων Μηχανικής Μάθησης με Δεδομένα από το OpenML

Τσαγκανός Αλέξανδρος

1 Φεβρουαρίου 2026

1 Τεχνική Αναφορά

1.1 Εισαγωγή

Σκοπός της παρούσας εργασίας είναι η πειραματική σύγκριση πέντε (5) κλασικών αλγορίθμων επιβλεπόμενης μάθησης (Supervised Learning) σε προβλήματα ταξινόμησης. Για τις ανάγκες του πειράματος χρησιμοποιήθηκαν τρία σύνολα δεδομένων (datasets) διαφορετικής πολυπλοκότητας και μεγέθους από το αποθετήριο OpenML: **Breast Cancer**, **Credit-G**, και **Adult**. Η υλοποίηση πραγματοποιήθηκε σε Python με τη χρήση της βιβλιοθήκης `scikit-learn`.

1.2 Μεθοδολογία

1.2.1 Περιγραφή Δεδομένων

- **Breast Cancer:** Μικρό dataset (286 δείγματα) με ιατρικά δεδομένα για την πρόβλεψη κακοήθειας.
- **Credit-G:** Μεσαίο dataset (1000 δείγματα) που αφορά την πιστοληπτική ικανότητα, με μίξη αριθμητικών και κατηγορικών χαρακτηριστικών.
- **Adult (Census Income):** Μεγάλο dataset (48.842 δείγματα) για την πρόβλεψη εισοδήματος ($> 50K / \leq 50K$), με έντονη ανισορροπία κλάσεων και υψηλή διαστασιμότητα.

1.2.2 Προεπεξεργασία (Preprocessing)

Λόγω της διαφορετικής φύσης των δεδομένων, εφαρμόστηκε μια ενιαία ροή προεπεξεργασίας:

- **Κωδικοποίηση (Encoding):** Τα κατηγορικά χαρακτηριστικά μετατράπηκαν σε αριθμητικά μέσω της μεθόδου **One-Hot Encoding** (`pd.get_dummies`). Αυτό αύξησε σημαντικά τον αριθμό των χαρακτηριστικών (features), ειδικά στο Adult dataset.
- **Διαχείριση Ελλιπών Τιμών:** Οι ελλιπείς τιμές αντικαταστάθηκαν με το μηδέν (0) για την απρόσκοπτη λειτουργία των αλγορίθμων.

- **Κανονικοποίηση (Scaling):** Εφαρμόστηκε **StandardScaler** (αφαίρεση μέσης τιμής και διαίρεση με τυπική απόκλιση) στο σύνολο των δεδομένων. Το βήμα αυτό κρίθηκε απαραίτητο για τη σωστή λειτουργία των αλγορίθμων που βασίζονται σε αποστάσεις (KNN, SVM) και του Νευρωνικού Δικτύου (MLP).

1.2.3 Αλγόριθμοι

Εξετάστηκαν οι εξής αλγόριθμοι:

1. **k-Nearest Neighbors (KNN)** ($k = 5$)
2. **Decision Tree (DT)** ($max_depth = 10$)
3. **Random Forest (RF)** ($n_estimators = 50$)
4. **Multi-Layer Perceptron (MLP)** (1 hidden layer, 50 neurons)
5. **Support Vector Machine (SVM)** (RBF kernel)

2 Αποτελέσματα Πειραμάτων

Παρακάτω παρουσιάζονται τα συγκεντρωτικά αποτελέσματα όπως προέκυψαν από την εκτέλεση του κώδικα (Cross-Validation).

2.1 Dataset: Breast Cancer

Algorithm	CV Acc	Precision	Recall	F1-Score
KNN	0.697	0.631	0.672	0.634
DT	0.697	0.695	0.707	0.699
RF	0.728	0.706	0.724	0.706
MLP	0.689	0.732	0.741	0.734
SVM	0.697	0.756	0.741	0.686

Πίνακας 1: Αποτελέσματα για Breast Cancer

2.2 Dataset: Credit-G

Algorithm	CV Acc	Precision	Recall	F1-Score
KNN	0.734	0.709	0.735	0.716
DT	0.684	0.683	0.685	0.684
RF	0.731	0.762	0.780	0.763
MLP	0.750	0.728	0.740	0.733
SVM	0.748	0.755	0.775	0.754

Πίνακας 2: Αποτελέσματα για Credit-G

2.3 Dataset: Adult

Algorithm	CV Acc	Precision	Recall	F1-Score
KNN	0.833	0.823	0.828	0.825
DT	0.845	0.834	0.842	0.834
RF	0.832	0.823	0.828	0.825
MLP	0.833	0.833	0.834	0.833
SVM	0.853	0.842	0.849	0.842

Πίνακας 3: Αποτελέσματα για Adult

3 Ανάλυση και Συζήτηση

3.1 Ποιος αλγόριθμος παρουσίασε την καλύτερη απόδοση και γιατί·

Τα αποτελέσματα διέφεραν ανάλογα με το μέγεθος και την πολυπλοκότητα του dataset:

- **Breast Cancer (Μικρό dataset):** Ο **Random Forest (RF)** πέτυχε την καλύτερη ισορροπία (Accuracy 0.728). Λόγω του μικρού αριθμού δειγμάτων, αλγόριθμοι όπως το MLP δυσκολεύτηκαν να γενικεύσουν.
- **Credit-G (Μεσαίο dataset):** Εδώ είχαμε ανταγωνισμό μεταξύ **SVM** και **Random Forest**. Αν και το SVM είχε ελαφρώς καλύτερη ακρίβεια, το Random Forest πέτυχε το υψηλότερο Recall (0.780), το οποίο είναι κρίσιμο σε προβλήματα πιστωτικού κινδύνου.
- **Adult (Μεγάλο dataset):** Το **SVM** ήταν ο ξεκάθαρος νικητής (Accuracy 0.853), αποδεικνύοντας ότι όταν υπάρχουν πολλά δεδομένα εκπαίδευσης, μπορεί να βρει το βέλτιστο διαχωριστικό υπερεπίπεδο.

3.2 Πώς επηρέασε η προεπεξεργασία τα αποτελέσματα·

- **Scaling (StandardScaler):** Ήταν καθοριστικό για την καλή απόδοση του **SVM**. Χωρίς αυτό, το SVM δεν θα μπορούσε να συγκλίνει σε λογικό χρόνο.
- **One-Hot Encoding:** Επέτρεψε τη χρήση όλων των χαρακτηριστικών, αλλά αύξησε τη πολυπλοκότητα. Αυτό φάνηκε στο **MLP**, το οποίο εμφάνισε παντού *Convergence Warnings*. Η αύξηση των διαστάσεων έκανε το δίκτυο να χρειάζεται περισσότερες εποχές για να εκπαιδευτεί.

3.3 Ποιοι αλγόριθμοι είναι πιο ευαίσθητοι στις υπερπαραμέτρους·

- **MLP:** Αποδείχθηκε το πιο ευαίσθητο. Το μοντέλο δεν πρόλαβε να συγκλίνει (max iterations reached). Χρειάζόταν αύξηση του `max_iter` ή ρύθμιση του `learning_rate`.
- **Decision Tree:** Στο `credit-g` είχε τη χειρότερη απόδοση (0.684), δείχνοντας ευαισθησία στο θόρυβο και τάση για overfitting.

3.4 Πλεονεκτήματα και Μειονεκτήματα ανά Dataset

Dataset	Αλγόριθμος	Πλεονεκτήματα	Μειονεκτήματα
Breast Cancer	Random Forest MLP	Σταθερότητα σε μικρά δεδομένα -	- Απέτυχε να συγκλίνει (Warnings)
Credit-G	Random Forest Decision Tree	Υψηλό Recall (ασφάλεια) Γρήγορο	- Χαμηλή ακρίβεια (Overfitting)
Adult	SVM KNN	Υψηλότερη ακρίβεια Αξιοπρεπής απόδοση	Πολύ αργό στην εκπαίδευση Αργό στο prediction phase

Πίνακας 4: Συγκριτικός Πίνακας Πλεονεκτημάτων

4 Παράρτημα: Κώδικας Python

Ακολουθεί ο κώδικας που χρησιμοποιήθηκε για τα πειράματα.

```
1 import pandas as pd
2 import numpy as np
3
4 from sklearn.datasets import fetch_openml
5 from sklearn.model_selection import train_test_split, cross_val_score
6 from sklearn.preprocessing import StandardScaler, LabelEncoder
7 from sklearn.metrics import precision_score, recall_score, f1_score,
   confusion_matrix
8
9 from sklearn.neural_network import MLPClassifier
10 from sklearn.neighbors import KNeighborsClassifier
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.svm import SVC
14
15
16 # datasets I want to compare on
17 # picked these mostly because they are common and not too huge
18 datasets = ['breast-cancer', 'credit-g', 'adult']
19
20
21 for ds_name in datasets:
22     print("\n=====")
23     print(f"Running experiments on: {ds_name}")
24     print("=====")
25
26     # grab data from openml
27     # parser='auto' avoids some annoying warnings in newer sklearn
28     X, y = fetch_openml(
29         ds_name,
30         version=1,
31         return_X_y=True,
32         as_frame=True,
33         parser='auto'
34     )
35
36     print("Original shape:", X.shape)
```

```

38 # --- preprocessing ---
39 # NOTE: this is intentionally very basic
40 # goal here is quick comparisons, not a production pipeline
41
42 # one-hot encode everything categorical
43 # yeah, this can blow up dimensionality but it is fine for now
44 X = pd.get_dummies(X)
45
46 # missing values:
47 # TODO: try mean/median imputation and see if it matters
48 X = X.fillna(0)
49
50 # scale features (needed for SVM / MLP)
51 scaler = StandardScaler()
52 X_scaled = scaler.fit_transform(X)
53
54 # encode labels if needed
55 le = LabelEncoder()
56 y_enc = le.fit_transform(y)
57
58 # train / test split
59 # no random_state on purpose - I want to see some variance
60 X_train, X_test, y_train, y_test = train_test_split(
61     X_scaled, y_enc, test_size=0.2
62 )
63
64 # models to try
65 # hyperparams are pretty arbitrary right now
66 models = [
67     ('KNN', KNeighborsClassifier(n_neighbors=5)),
68     ('DT', DecisionTreeClassifier(max_depth=10)),
69     ('RF', RandomForestClassifier(n_estimators=50)),
70     ('MLP', MLPClassifier(hidden_layer_sizes=(50,), max_iter=300)),
71     ('SVM', SVC())
72 ]
73
74 results = []
75
76 for model_name, model in models:
77     print(f"\nTraining {model_name}...")
78
79     # quick CV just to get a rough accuracy estimate
80     # cv=3 to keep runtime reasonable
81     cv_acc = cross_val_score(
82         model,
83         X_train,
84         y_train,
85         cv=3,
86         scoring='accuracy'
87     ).mean()
88
89     # fit on full training set
90     model.fit(X_train, y_train)
91
92     # evaluate on test set
93     preds = model.predict(X_test)
94
95     # weighted metrics since some datasets are imbalanced

```

```

96     prec = precision_score(
97         y_test, preds, average='weighted', zero_division=0
98     )
99     rec = recall_score(
100         y_test, preds, average='weighted', zero_division=0
101     )
102     f1 = f1_score(
103         y_test, preds, average='weighted', zero_division=0
104     )
105
106     # just eyeballing confusion matrices for now
107     cm = confusion_matrix(y_test, preds)
108     # print(cm) # uncomment if needed for debugging
109
110     results.append({
111         'Model': model_name,
112         'CV_Accuracy': cv_acc,
113         'Precision': prec,
114         'Recall': rec,
115         'F1': f1
116     })
117
118     # summarize results for this dataset
119     print("\nSummary:")
120     results_df = pd.DataFrame(results)
121     print(results_df.round(3))

```